

Module - II 12 Hrs

Multi Threading: Java Thread Model, Thread Priorities, Synchronization, Creating a thread, Creating Multiple threads, Using `isAlive ( )` and `join ( )`, `wait ( )` & `notify ( )`.

String Handling: String constructors, String length, Character Extraction, String Comparison, Modifying a string.

Java I/O: Classes & Interfaces, Stream classes, Byte streams, Character streams, Serialization.

JDBC: Fundamentals, Type I, Type II, Type III, Type IV drivers.

Networking: Basics, Socket overview, Networking classes, & interfaces, TCP/IP client sockets, `whois`, URL format, `URLConnection`, TCP/IP Server Sockets.

CHAPTER 14

MULTITHREADING

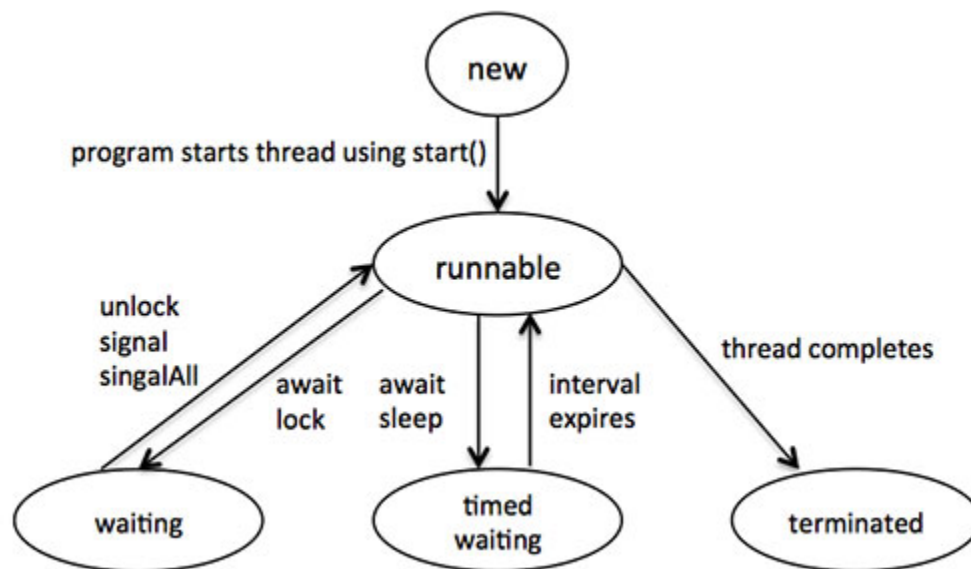
Java is a *multithreaded programming language* which means we can develop multithreaded program using Java. A multithreaded program contains two or more parts that can run concurrently and each part can handle different task at the same time making optimal use of the available resources specially when your computer has multiple CPUs.

By definition multitasking is when multiple processes share common processing resources such as a CPU. Multithreading extends the idea of multitasking into applications where you can subdivide specific operations within a single application into individual threads. Each of the threads can run in parallel. The OS divides processing time not only among different applications, but also among each thread within an application.

Multithreading enables you to write in a way where multiple activities can proceed concurrently in the same program.

Life Cycle of a Thread:

A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies. Following diagram shows complete life cycle of a thread.



Above-mentioned stages are explained here:

- **New:** A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a born thread.
- **Runnable:** After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting:** Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

- **Timed waiting:** A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.
- **Terminated:** A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Thread Priorities:

Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled.

Java thread priorities are in the range between `MIN_PRIORITY` (a constant of 1) and `MAX_PRIORITY` (a constant of 10). By default, every thread is given priority `NORM_PRIORITY` (a constant of 5).

Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads. However, thread priorities cannot guarantee the order in which threads execute and very much platform dependant.

Create Thread by Implementing Runnable Interface:

If your class is intended to be executed as a thread then you can achieve this by implementing **Runnable** interface. You will need to follow three basic steps:

Step 1:

As a first step you need to implement a `run()` method provided by **Runnable** interface. This method provides entry point for the thread and you will put your complete business logic inside this method. Following is simple syntax of `run()` method:

```
public void run( )
```

Step 2:

At second step you will instantiate a **Thread** object using the following constructor:

```
Thread(Runnable threadObj, String threadName);
```

Where, *threadObj* is an instance of a class that implements the **Runnable** interface and **threadName** is the name given to the new thread.

Step 3

Once Thread object is created, you can start it by calling **start()** method, which executes a call to `run()` method. Following is simple syntax of `start()` method:

```
void start( );
```

Example:

Here is an example that creates a new thread and starts it running:

```
class RunnableDemo implements Runnable {
```

```

private Thread t;
private String threadName;

RunnableDemo( String name){
    threadName = name;
    System.out.println("Creating " + threadName );
}
public void run() {
    System.out.println("Running " + threadName );
    try {
        for(int i = 4; i > 0; i--) {
            System.out.println("Thread: " + threadName + ", " + i);
            // Let the thread sleep for a while.
            Thread.sleep(50);
        }
    } catch (InterruptedException e) {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}

public void start ()
{
    System.out.println("Starting " + threadName );
    if (t == null)
    {
        t = new Thread (this, threadName);
        t.start ();
    }
}

}

public class TestThread {
    public static void main(String args[]) {

        RunnableDemo R1 = new RunnableDemo( "Thread-1");
        R1.start();

        RunnableDemo R2 = new RunnableDemo( "Thread-2");
        R2.start();
    }
}

```

This would produce the following result:

```

Creating Thread-1
Starting Thread-1
Creating Thread-2
Starting Thread-2
Running Thread-1
Thread: Thread-1, 4
Running Thread-2
Thread: Thread-2, 4
Thread: Thread-1, 3

```

Thread: Thread-2, 3
Thread: Thread-1, 2
Thread: Thread-2, 2
Thread: Thread-1, 1
Thread: Thread-2, 1
Thread Thread-1 exiting.
Thread Thread-2 exiting.

Create Thread by Extending Thread Class:

The second way to create a thread is to create a new class that extends **Thread** class using the following two simple steps. This approach provides more flexibility in handling multiple threads created using available methods in Thread class.

Step 1

You will need to override **run()** method available in Thread class. This method provides entry point for the thread and you will put your complete business logic inside this method. Following is simple syntax of run() method:

```
public void run( )
```

Step 2

Once Thread object is created, you can start it by calling **start()** method, which executes a call to run() method. Following is simple syntax of start() method:

```
void start( );
```

Example:

Here is the preceding program rewritten to extend Thread:

```
class ThreadDemo extends Thread {  
    private Thread t;  
    private String threadName;  
  
    ThreadDemo( String name){  
        threadName = name;  
        System.out.println("Creating " + threadName );  
    }  
    public void run() {  
        System.out.println("Running " + threadName );  
        try {  
            for(int i = 4; i > 0; i--) {  
                System.out.println("Thread: " + threadName + ", " + i);  
                // Let the thread sleep for a while.  
                Thread.sleep(50);  
            }  
        } catch (InterruptedException e) {  
            System.out.println("Thread " + threadName + " interrupted.");  
        }  
        System.out.println("Thread " + threadName + " exiting.");  
    }  
}
```

```

public void start ()
{
    System.out.println("Starting " + threadName );
    if (t == null)
    {
        t = new Thread (this, threadName);
        t.start ();
    }
}

}

public class TestThread {
    public static void main(String args[]) {

        ThreadDemo T1 = new ThreadDemo( "Thread-1");
        T1.start();

        ThreadDemo T2 = new ThreadDemo( "Thread-2");
        T2.start();
    }
}

```

This would produce the following result:

```

Creating Thread-1
Starting Thread-1
Creating Thread-2
Starting Thread-2
Running Thread-1
Thread: Thread-1, 4
Running Thread-2
Thread: Thread-2, 4
Thread: Thread-1, 3
Thread: Thread-2, 3
Thread: Thread-1, 2
Thread: Thread-2, 2
Thread: Thread-1, 1
Thread: Thread-2, 1
Thread Thread-1 exiting.
Thread Thread-2 exiting.

```

Thread Methods:

Following is the list of important methods available in the Thread class.

SN	Methods with Description
1	public void start() Starts the thread in a separate path of execution, then invokes the run() method on this Thread object.
2	public void run() If this Thread object was instantiated using a separate Runnable target, the run() method is invoked on that Runnable object.

3	public final void setName(String name) Changes the name of the Thread object. There is also a getName() method for retrieving the name.
4	public final void setPriority(int priority) Sets the priority of this Thread object. The possible values are between 1 and 10.
5	public final void setDaemon(boolean on) A parameter of true denotes this Thread as a daemon thread.
6	public final void join(long millisec) The current thread invokes this method on a second thread, causing the current thread to block until the second thread terminates or the specified number of milliseconds passes.
7	public void interrupt() Interrupts this thread, causing it to continue execution if it was blocked for any reason.
8	public final boolean isAlive() Returns true if the thread is alive, which is any time after the thread has been started but before it runs to completion.

The previous methods are invoked on a particular Thread object. The following methods in the Thread class are static. Invoking one of the static methods performs the operation on the currently running thread.

SN	Methods with Description
1	public static void yield() Causes the currently running thread to yield to any other threads of the same priority that are waiting to be scheduled.
2	public static void sleep(long millisec) Causes the currently running thread to block for at least the specified number of milliseconds.
3	public static boolean holdsLock(Object x) Returns true if the current thread holds the lock on the given Object.
4	public static Thread currentThread() Returns a reference to the currently running thread, which is the thread that invokes this method.
5	public static void dumpStack() Prints the stack trace for the currently running thread, which is useful when debugging a multithreaded application.

Example:

The following ThreadClassDemo program demonstrates some of these methods of the Thread class. Consider a class **DisplayMessage** which implements **Runnable**:

```
// File Name : DisplayMessage.java
// Create a thread to implement Runnable
public class DisplayMessage implements Runnable
{
    private String message;
    public DisplayMessage(String message)
    {
        this.message = message;
    }
    public void run()
    {
        while(true)
        {
```

```

        System.out.println(message);
    }
}
}

```

Following is another class which extends Thread class:

```

// File Name : GuessANumber.java
// Create a thread to extend Thread
public class GuessANumber extends Thread
{
    private int number;
    public GuessANumber(int number)
    {
        this.number = number;
    }
    public void run()
    {
        int counter = 0;
        int guess = 0;
        do
        {
            guess = (int) (Math.random() * 100 + 1);
            System.out.println(this.getName()
                               + " guesses " + guess);
            counter++;
        } while(guess != number);
        System.out.println("** Correct! " + this.getName()
                           + " in " + counter + " guesses. **");
    }
}

```

Following is the main program which makes use of above defined classes:

```

// File Name : ThreadClassDemo.java
public class ThreadClassDemo
{
    public static void main(String [] args)
    {
        Runnable hello = new DisplayMessage("Hello");
        Thread thread1 = new Thread(hello);
        thread1.setDaemon(true);
        thread1.setName("hello");
        System.out.println("Starting hello thread...");
        thread1.start();

        Runnable bye = new DisplayMessage("Goodbye");
        Thread thread2 = new Thread(bye);
        thread2.setPriority(Thread.MIN_PRIORITY);
        thread2.setDaemon(true);
        System.out.println("Starting goodbye thread...");
        thread2.start();

        System.out.println("Starting thread3...");
        Thread thread3 = new GuessANumber(27);
    }
}

```



```

        thread3.start();
    try
    {
        thread3.join();
    } catch (InterruptedException e)
    {
        System.out.println("Thread interrupted.");
    }
    System.out.println("Starting thread4...");
    Thread thread4 = new GuessANumber(75);

        thread4.start();
    System.out.println("main() is ending...");
    }
}

```

This would produce the following result. You can try this example again and again and you would get different result every time.

```

Starting hello thread...
Starting goodbye thread...
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Goodbye
Goodbye
Goodbye
Goodbye
Goodbye
.....

```

Thread Synchronization:-

When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issue. For example if multiple threads try to write within a same file then they may corrupt the data because one of the threads can overwrite data or while one thread is opening the same file at the same time another thread might be closing the same file.

So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time. This is implemented using a concept called **monitors**. Each object in Java is associated with a monitor, which a thread can lock or unlock. Only one thread at a time may hold a lock on a monitor.

Java programming language provides a very handy way of creating threads and synchronizing their task by using **synchronized** blocks. You keep shared resources within this block. Following is the general form of the synchronized statement:

```

synchronized(objectidentifier) {
    // Access shared variables and other shared resources
}

```

Here, the **objectidentifier** is a reference to an object whose lock associates with the monitor that the synchronized statement represents. Now we are going to see two examples where we will print a counter using two different threads. When threads are not synchronized, they print counter value which is not in sequence, but when we print counter by putting inside synchronized() block, then it prints counter very much in sequence for both the threads.

Multithreading example without Synchronization:

Here is a simple example which may or may not print counter value in sequence and every time we run it, it produces different result based on CPU availability to a thread.

```
class PrintDemo {
    public void printCount(){
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Counter --- " + i );
            }
        } catch (Exception e) {
            System.out.println("Thread interrupted.");
        }
    }
}

class ThreadDemo extends Thread {
    private Thread t;
    private String threadName;
    PrintDemo PD;

    ThreadDemo( String name, PrintDemo pd){
        threadName = name;
        PD = pd;
    }
    public void run() {
        PD.printCount();
        System.out.println("Thread " + threadName + " exiting.");
    }

    public void start ()
    {
        System.out.println("Starting " + threadName );
        if (t == null)
        {
            t = new Thread (this, threadName);
            t.start ();
        }
    }
}

public class TestThread {
    public static void main(String args[]) {

        PrintDemo PD = new PrintDemo();

        ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
```

```

ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

T1.start();
T2.start();

// wait for threads to end
try {
    T1.join();
    T2.join();
} catch( Exception e) {
    System.out.println("Interrupted");
}
}
}

```

This produces different result every time you run this program:

```

Starting Thread - 1
Starting Thread - 2
Counter --- 5
Counter --- 4
Counter --- 3
Counter --- 5
Counter --- 2
Counter --- 1
Counter --- 4
Thread Thread - 1 exiting.
Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 2 exiting.

```

Multithreading example with Synchronization:

Here is the same example which prints counter value in sequence and every time we run it, it produces same result.

```

class PrintDemo {
    public void printCount(){
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Counter --- " + i);
            }
        } catch (Exception e) {
            System.out.println("Thread interrupted.");
        }
    }
}

class ThreadDemo extends Thread {
    private Thread t;
    private String threadName;
    PrintDemo PD;

    ThreadDemo( String name, PrintDemo pd){

```

```

        threadName = name;
        PD = pd;
    }
    public void run() {
        synchronized(PD) {
            PD.printCount();
        }
        System.out.println("Thread " + threadName + " exiting.");
    }

    public void start ()
    {
        System.out.println("Starting " + threadName );
        if (t == null)
        {
            t = new Thread (this, threadName);
            t.start ();
        }
    }
}

public class TestThread {
    public static void main(String args[]) {

        PrintDemo PD = new PrintDemo();

        ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
        ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

        T1.start();
        T2.start();

        // wait for threads to end
        try {
            T1.join();
            T2.join();
        } catch( Exception e) {
            System.out.println("Interrupted");
        }
    }
}

```

This produces same result every time you run this program:

```

Starting Thread - 1
Starting Thread - 2
Counter --- 5
Counter --- 4
Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 1 exiting.
Counter --- 5
Counter --- 4

```

```
Counter --- 3
Counter --- 2
Counter --- 1
Thread Thread - 2 exiting.
```

Thread Deadlock:-

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other. Deadlock occurs when multiple threads need the same locks but obtain them in different order. A Java multithreaded program may suffer from the deadlock condition because the **synchronized** keyword causes the executing thread to block while waiting for the lock, or monitor, associated with the specified object. Here is an example:

Example:

```
public class TestThread {
    public static Object Lock1 = new Object();
    public static Object Lock2 = new Object();

    public static void main(String args[]) {

        ThreadDemo1 T1 = new ThreadDemo1();
        ThreadDemo2 T2 = new ThreadDemo2();
        T1.start();
        T2.start();
    }

    private static class ThreadDemo1 extends Thread {
        public void run() {
            synchronized (Lock1) {
                System.out.println("Thread 1: Holding lock 1...");
                try { Thread.sleep(10); }
                catch (InterruptedException e) {}
                System.out.println("Thread 1: Waiting for lock 2...");
                synchronized (Lock2) {
                    System.out.println("Thread 1: Holding lock 1 & 2...");
                }
            }
        }
    }

    private static class ThreadDemo2 extends Thread {
        public void run() {
            synchronized (Lock2) {
                System.out.println("Thread 2: Holding lock 2...");
                try { Thread.sleep(10); }
                catch (InterruptedException e) {}
                System.out.println("Thread 2: Waiting for lock 1...");
                synchronized (Lock1) {
                    System.out.println("Thread 2: Holding lock 1 & 2...");
                }
            }
        }
    }
}
```

When you compile and execute above program, you find a deadlock situation and below is the output produced by the program:

```
Thread 1: Holding lock 1...
Thread 2: Holding lock 2...
Thread 1: Waiting for lock 2...
Thread 2: Waiting for lock 1...
```

Above program will hang forever because neither of the threads in position to proceed and waiting for each other to release the lock, so you can come out of the program by pressing CTRL-C.

Deadlock Solution Example:

Let's change the order of the lock and run the same program to see if still both the threads waits for each other:

```
public class TestThread {
    public static Object Lock1 = new Object();
    public static Object Lock2 = new Object();

    public static void main(String args[]) {

        ThreadDemo1 T1 = new ThreadDemo1();
        ThreadDemo2 T2 = new ThreadDemo2();
        T1.start();
        T2.start();
    }

    private static class ThreadDemo1 extends Thread {
        public void run() {
            synchronized (Lock1) {
                System.out.println("Thread 1: Holding lock 1...");
                try { Thread.sleep(10); }
                catch (InterruptedException e) {}
                System.out.println("Thread 1: Waiting for lock 2...");
                synchronized (Lock2) {
                    System.out.println("Thread 1: Holding lock 1 & 2...");
                }
            }
        }
    }

    private static class ThreadDemo2 extends Thread {
        public void run() {
            synchronized (Lock1) {
                System.out.println("Thread 2: Holding lock 1...");
                try { Thread.sleep(10); }
                catch (InterruptedException e) {}
                System.out.println("Thread 2: Waiting for lock 2...");
                synchronized (Lock2) {
                    System.out.println("Thread 2: Holding lock 1 & 2...");
                }
            }
        }
    }
}
```

So just changing the order of the locks prevent the program in going deadlock situation and completes with the following result:

Thread 1: Holding lock 1...
Thread 1: Waiting for lock 2...
Thread 1: Holding lock 1 & 2...
Thread 2: Holding lock 1...
Thread 2: Waiting for lock 2...
Thread 2: Holding lock 1 & 2...

Above example has been shown just for making you the concept clear, but its a more complex concept and you should deep dive into it before you develop your applications to deal with deadlock situations.

Thread Control:-

Core Java provides a complete control over multithreaded program. You can develop a multithreaded program which can be suspended, resumed or stopped completely based on your requirements. There are various static methods which you can use on thread objects to control their behavior. Following table lists down those methods:

SN	Methods with Description
1	public void suspend() This method puts a thread in suspended state and can be resumed using resume() method.
2	public void stop() This method stops a thread completely.
3	public void resume() This method resumes a thread which was suspended using suspend() method.
4	public void wait() Causes the current thread to wait until another thread invokes the notify().
5	public void notify() Wakes up a single thread that is waiting on this object's monitor.

Be aware that latest versions of Java has deprecated the usage of suspend(), resume(), and stop() methods and so you need to use available alternatives.

Example:

```
class RunnableDemo implements Runnable {  
    public Thread t;  
    private String threadName;  
    boolean suspended = false;  
  
    RunnableDemo( String name){  
        threadName = name;  
        System.out.println("Creating " + threadName );  
    }  
}
```

```

public void run() {
    System.out.println("Running " + threadName );
    try {
        for(int i = 10; i > 0; i--) {
            System.out.println("Thread: " + threadName + ", " + i);
            // Let the thread sleep for a while.
            Thread.sleep(300);
            synchronized(this) {
                while(suspended) {
                    wait();
                }
            }
        }
    } catch (InterruptedException e) {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}

public void start ()
{
    System.out.println("Starting " + threadName );
    if (t == null)
    {
        t = new Thread (this, threadName);
        t.start ();
    }
}

void suspend() {
    suspended = true;
}

synchronized void resume() {
    suspended = false;
    notify();
}
}

```

```

public class TestThread {
    public static void main(String args[]) {

        RunnableDemo R1 = new RunnableDemo( "Thread-1");
        R1.start();

        RunnableDemo R2 = new RunnableDemo( "Thread-2");
        R2.start();

        try {
            Thread.sleep(1000);
            R1.suspend();
            System.out.println("Suspending First Thread");
            Thread.sleep(1000);
            R1.resume();
            System.out.println("Resuming First Thread");
            R2.suspend();
            System.out.println("Suspending thread Two");
            Thread.sleep(1000);

```



```

        R2.resume();
        System.out.println("Resuming thread Two");
    } catch (InterruptedException e) {
        System.out.println("Main thread Interrupted");
    }
    try {
        System.out.println("Waiting for threads to finish.");
        R1.t.join();
        R2.t.join();
    } catch (InterruptedException e) {
        System.out.println("Main thread Interrupted");
    }
    System.out.println("Main thread exiting.");
}
}

```

Here is the output produced by the above program:

```

Creating Thread-1
Starting Thread-1
Creating Thread-2
Starting Thread-2
Running Thread-1
Thread: Thread-1, 10
Running Thread-2
Thread: Thread-2, 10
Thread: Thread-1, 9
Thread: Thread-2, 9
Thread: Thread-1, 8
Thread: Thread-2, 8
Thread: Thread-1, 7
Thread: Thread-2, 7
Suspending First Thread
Thread: Thread-2, 6
Thread: Thread-2, 5
Thread: Thread-2, 4
Resuming First Thread
Suspending thread Two
Thread: Thread-1, 6
Thread: Thread-1, 5
Thread: Thread-1, 4
Thread: Thread-1, 3
Resuming thread Two
Thread: Thread-2, 3
Waiting for threads to finish.
Thread: Thread-1, 2
Thread: Thread-2, 2
Thread: Thread-1, 1
Thread: Thread-2, 1
Thread Thread-1 exiting.
Thread Thread-2 exiting.
Main thread exiting.

```

CHAPTER 15

STRING HANDLING

Strings, which are widely used in Java programming, are a sequence of characters. In the Java programming language, strings are objects.

The Java platform provides the String class to create and manipulate strings.

Creating Strings:

The most direct way to create a string is to write:

```
String greeting = "Hello world!";
```

Whenever it encounters a string literal in your code, the compiler creates a String object with its value in this case, "Hello world!".

As with any other object, you can create String objects by using the new keyword and a constructor. The String class has eleven constructors that allow you to provide the initial value of the string using different sources, such as an array of characters.

```
public class StringDemo{  
  
    public static void main(String args[]){  
        char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.' };  
        String helloString = new String(helloArray);  
        System.out.println( helloString );  
    }  
}
```

This would produce the following result:

hello.

Note: The String class is immutable, so that once it is created a String object cannot be changed. If there is a necessity to make a lot of modifications to Strings of characters, then you should use [String Buffer & String Builder](#) Classes.

String Length:

Methods used to obtain information about an object are known as accessor methods. One accessor method that you can use with strings is the length() method, which returns the number of characters contained in the string object.

After the following two lines of code have been executed, len equals 17:

```
public class StringDemo {  
  
    public static void main(String args[]) {  
        String palindrome = "Dot saw I was Tod";
```

```

        int len = palindrome.length();
        System.out.println( "String Length is : " + len );
    }
}

```

This would produce the following result:

String Length is : 17

Concatenating Strings:

The String class includes a method for concatenating two strings:

```
string1.concat(string2);
```

This returns a new string that is string1 with string2 added to it at the end. You can also use the concat() method with string literals, as in:

```
"My name is ".concat("Zara");
```

Strings are more commonly concatenated with the + operator, as in:

```
"Hello," + " world" + "!"
```

which results in:

```
"Hello, world!"
```

Let us look at the following example:

```

public class StringDemo {

    public static void main(String args[]) {
        String string1 = "saw I was ";
        System.out.println("Dot " + string1 + "Tod");
    }
}

```

This would produce the following result:

Dot saw I was Tod

Creating Format Strings:

You have printf() and format() methods to print output with formatted numbers. The String class has an equivalent class method, format(), that returns a String object rather than a PrintStream object.

Using String's static format() method allows you to create a formatted string that you can reuse, as opposed to a one-time print statement. For example, instead of:

```
System.out.printf("The value of the float variable is " +
```

```

"%f, while the value of the integer " +
"variable is %d, and the string " +
"is %s", floatVar, intVar, stringVar);

```

you can write:

```

String fs;
fs = String.format("The value of the float variable is " +
"%f, while the value of the integer " +
"variable is %d, and the string " +
"is %s", floatVar, intVar, stringVar);
System.out.println(fs);

```

String Methods:

Here is the list of methods supported by String class:

SN	Methods with Description
1	char charAt(int index) Returns the character at the specified index.
2	int compareTo(Object o) Compares this String to another Object.
3	int compareTo(String anotherString) Compares two strings lexicographically.
4	int compareToIgnoreCase(String str) Compares two strings lexicographically, ignoring case differences.
5	String concat(String str) Concatenates the specified string to the end of this string.
6	boolean contentEquals(StringBuffer sb) Returns true if and only if this String represents the same sequence of characters as the specified StringBuffer.
7	static String copyValueOf(char[] data) Returns a String that represents the character sequence in the array specified.
8	static String copyValueOf(char[] data, int offset, int count) Returns a String that represents the character sequence in the array specified.
9	boolean endsWith(String suffix) Tests if this string ends with the specified suffix.
10	boolean equals(Object anObject) Compares this string to the specified object.
11	boolean equalsIgnoreCase(String anotherString) Compares this String to another String, ignoring case considerations.
12	byte getBytes() Encodes this String into a sequence of bytes using the platform's default charset, storing the result into a new byte array.
13	byte[] getBytes(String charsetName) Encodes this String into a sequence of bytes using the named charset, storing the result into a new byte array.
14	void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin) Copies characters from this string into the destination character array.
15	int hashCode() Returns a hash code for this string.

16	<u>int indexOf(int ch)</u> Returns the index within this string of the first occurrence of the specified character.
17	<u>int indexOf(int ch, int fromIndex)</u> Returns the index within this string of the first occurrence of the specified character, starting the search at the specified index.
18	<u>int indexOf(String str)</u> Returns the index within this string of the first occurrence of the specified substring.
19	<u>int indexOf(String str, int fromIndex)</u> Returns the index within this string of the first occurrence of the specified substring, starting at the specified index.
20	<u>String intern()</u> Returns a canonical representation for the string object.
21	<u>int lastIndexOf(int ch)</u> Returns the index within this string of the last occurrence of the specified character.
22	<u>int lastIndexOf(int ch, int fromIndex)</u> Returns the index within this string of the last occurrence of the specified character, searching backward starting at the specified index.
23	<u>int lastIndexOf(String str)</u> Returns the index within this string of the rightmost occurrence of the specified substring.
24	<u>int lastIndexOf(String str, int fromIndex)</u> Returns the index within this string of the last occurrence of the specified substring, searching backward starting at the specified index.
25	<u>int length()</u> Returns the length of this string.
26	<u>boolean matches(String regex)</u> Tells whether or not this string matches the given regular expression.
27	<u>boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)</u> Tests if two string regions are equal.
28	<u>boolean regionMatches(int toffset, String other, int ooffset, int len)</u> Tests if two string regions are equal.
29	<u>String replace(char oldChar, char newChar)</u> Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.
30	<u>String replaceAll(String regex, String replacement)</u> Replaces each substring of this string that matches the given regular expression with the given replacement.
31	<u>String replaceFirst(String regex, String replacement)</u> Replaces the first substring of this string that matches the given regular expression with the given replacement.
32	<u>String[] split(String regex)</u> Splits this string around matches of the given regular expression.
33	<u>String[] split(String regex, int limit)</u> Splits this string around matches of the given regular expression.
34	<u>boolean startsWith(String prefix)</u> Tests if this string starts with the specified prefix.
35	<u>boolean startsWith(String prefix, int toffset)</u> Tests if this string starts with the specified prefix beginning a specified index.
36	<u>CharSequence subSequence(int beginIndex, int endIndex)</u> Returns a new character sequence that is a subsequence of this sequence.
37	<u>String substring(int beginIndex)</u> Returns a new string that is a substring of this string.

38	<u>String substring(int beginIndex, int endIndex)</u> Returns a new string that is a substring of this string.
39	<u>char[] toCharArray()</u> Converts this string to a new character array.
40	<u>String toLowerCase()</u> Converts all of the characters in this String to lower case using the rules of the default locale.
41	<u>String toLowerCase(Locale locale)</u> Converts all of the characters in this String to lower case using the rules of the given Locale.
42	<u>String toString()</u> This object (which is already a string!) is itself returned.
43	<u>String toUpperCase()</u> Converts all of the characters in this String to upper case using the rules of the default locale.
44	<u>String toUpperCase(Locale locale)</u> Converts all of the characters in this String to upper case using the rules of the given Locale.
45	<u>String trim()</u> Returns a copy of the string, with leading and trailing whitespace omitted.
46	<u>static String valueOf(primitive data type x)</u> Returns the string representation of the passed data type argument.

CHAPTER 16

JAVA FILES I/O

The java.io package contains nearly every class you might ever need to perform input and output (I/O) in Java. All these streams represent an input source and an output destination. The stream in the java.io package supports many data such as primitives, Object, localized characters, etc.

A stream can be defined as a sequence of data. The InputStream is used to read data from a source and the OutputStream is used for writing data to a destination.

Java provides strong but flexible support for I/O related to Files and networks but this tutorial covers very basic functionality related to streams and I/O. We would see most commonly used example one by one:

Byte Streams:-

Java byte streams are used to perform input and output of 8-bit bytes. Though there are many classes related to byte streams but the most frequently used classes are , **FileInputStream** and **FileOutputStream**. Following is an example which makes use of these two classes to copy an input file into an output file:

```
import java.io.*;

public class CopyFile {
    public static void main(String args[]) throws IOException
    {
        FileInputStream in = null;
        FileOutputStream out = null;

        try {
            in = new FileInputStream("input.txt");
            out = new FileOutputStream("output.txt");

            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

Now let's have a file **input.txt** with the following content:

This is test for copy file.

As a next step, compile above program and execute it, which will result in creating output.txt file with the same content as we have in input.txt. So let's put above code in CopyFile.java file and do the following:

```
$javac CopyFile.java
$java CopyFile
```

Character Streams:-

Java **Byte** streams are used to perform input and output of 8-bit bytes, where as Java **Character** streams are used to perform input and output for 16-bit unicode. Though there are many classes related to character streams but the most frequently used classes are , **FileReader** and **FileWriter**.. Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but here major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

We can re-write above example which makes use of these two classes to copy an input file (having unicode characters) into an output file:

```
import java.io.*;

public class CopyFile {
    public static void main(String args[]) throws IOException
    {
        FileReader in = null;
        FileWriter out = null;

        try {
            in = new FileReader("input.txt");
            out = new FileWriter("output.txt");

            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

Now let's have a file **input.txt** with the following content:

This is test for copy file.

As a next step, compile above program and execute it, which will result in creating output.txt file with the same content as we have in input.txt. So let's put above code in CopyFile.java file and do the following:

```
$javac CopyFile.java
$java CopyFile
```


Standard Streams:-

All the programming languages provide support for standard I/O where user's program can take input from a keyboard and then produce output on the computer screen. If you are aware of C or C++ programming languages, then you must be aware of three standard devices STDIN, STDOUT and STDERR. Similar way Java provides following three standard streams

- **Standard Input:** This is used to feed the data to user's program and usually a keyboard is used as standard input stream and represented as **System.in**.
- **Standard Output:** This is used to output the data produced by the user's program and usually a computer screen is used to standard output stream and represented as **System.out**.
- **Standard Error:** This is used to output the error data produced by the user's program and usually a computer screen is used to standard error stream and represented as **System.err**.

Following is a simple program which creates **InputStreamReader** to read standard input stream until the user types a "q":

```
import java.io.*;

public class ReadConsole {
    public static void main(String args[]) throws IOException
    {
        InputStreamReader cin = null;

        try {
            cin = new InputStreamReader(System.in);
            System.out.println("Enter characters, 'q' to quit.");
            char c;
            do {
                c = (char) cin.read();
                System.out.print(c);
            } while(c != 'q');
        } finally {
            if (cin != null) {
                cin.close();
            }
        }
    }
}
```

Let's keep above code in ReadConsole.java file and try to compile and execute it as below. This program continues reading and outputting same character until we press 'q':

```
$javac ReadConsole.java
$java ReadConsole
Enter characters, 'q' to quit.
l
l
e
e
q
q
```

Reading and Writing Files:

As described earlier, A stream can be defined as a sequence of data. The **InputStream** is used to read data from a source and the **OutputStream** is used for writing data to a destination.

Here is a hierarchy of classes to deal with Input and Output streams.

The two important streams are **FileInputStream** and **FileOutputStream**, which would be discussed in this tutorial:

FileInputStream:

This stream is used for reading data from the files. Objects can be created using the keyword new and there are several types of constructors available.

Following constructor takes a file name as a string to create an input stream object to read the file.:

```
InputStream f = new FileInputStream("C:/java/hello");
```

Following constructor takes a file object to create an input stream object to read the file. First we create a file object using File() method as follows:

```
File f = new File("C:/java/hello");  
InputStream f = new FileInputStream(f);
```

Once you have *InputStream* object in hand, then there is a list of helper methods which can be used to read to stream or to do other operations on the stream.

SN	Methods with Description
1	public void close() throws IOException{} This method closes the file output stream. Releases any system resources associated with the file. Throws an IOException.
2	protected void finalize()throws IOException {} This method cleans up the connection to the file. Ensures that the close method of this file output stream is called when there are no more references to this stream. Throws an IOException.
3	public int read(int r)throws IOException{} This method reads the specified byte of data from the InputStream. Returns an int. Returns the next byte of data and -1 will be returned if it's end of file.
4	public int read(byte[] r) throws IOException{} This method reads r.length bytes from the input stream into an array. Returns the total number of bytes read. If end of file -1 will be returned.
5	public int available() throws IOException{} Gives the number of bytes that can be read from this file input stream. Returns an int.

There are other important input streams available, for more detail you can refer to the following links:

- [ByteArrayInputStream](#)
- [DataInputStream](#)

FileOutputStream:

FileOutputStream is used to create a file and write data into it. The stream would create a file, if it doesn't already exist, before opening it for output.

Here are two constructors which can be used to create a FileOutputStream object.

Following constructor takes a file name as a string to create an input stream object to write the file:

```
OutputStream f = new FileOutputStream("C:/java/hello")
```

Following constructor takes a file object to create an output stream object to write the file. First, we create a file object using File() method as follows:

```
File f = new File("C:/java/hello");  
OutputStream f = new FileOutputStream(f);
```

Once you have *OutputStream* object in hand, then there is a list of helper methods, which can be used to write to stream or to do other operations on the stream.

SN	Methods with Description
1	public void close() throws IOException{} This method closes the file output stream. Releases any system resources associated with the file. Throws an IOException.
2	protected void finalize()throws IOException {} This method cleans up the connection to the file. Ensures that the close method of this file output stream is called when there are no more references to this stream. Throws an IOException.
3	public void write(int w)throws IOException{} This methods writes the specified byte to the output stream.
4	public void write(byte[] w) Writes w.length bytes from the mentioned byte array to the OutputStream.

There are other important output streams available, for more detail you can refer to the following links:

- [ByteArrayOutputStream](#)
- [DataOutputStream](#)

Example:

Following is the example to demonstrate InputStream and OutputStream:

```
import java.io.*;  
  
public class fileStreamTest{  
  
    public static void main(String args[]){  
  
        try{  
            byte bWrite [] = {11,21,3,40,5};  
            OutputStream os = new FileOutputStream("test.txt");  
            for(int x=0; x < bWrite.length ; x++){
```

```

        os.write( bWrite[x] ); // writes the bytes
    }
    os.close();

    InputStream is = new FileInputStream("test.txt");
    int size = is.available();

    for(int i=0; i< size; i++){
        System.out.print((char)is.read() + " ");
    }
    is.close();
} catch(IOException e){
    System.out.print("Exception");
}
}
}
}

```

The above code would create file test.txt and would write given numbers in binary format. Same would be output on the stdout screen.

File Navigation and I/O:

There are several other classes that we would be going through to get to know the basics of File Navigation and I/O.

- [File Class](#)
- [FileReader Class](#)
- [FileWriter Class](#)

Directories in Java:

A directory is a File which can contains a list of other files and directories. You use **File** object to create directories, to list down files available in a directory. For complete detail check a list of all the methods which you can call on File object and what are related to directories.

Creating Directories:

There are two useful **File** utility methods, which can be used to create directories:

- The **mkdir()** method creates a directory, returning true on success and false on failure. Failure indicates that the path specified in the File object already exists, or that the directory cannot be created because the entire path does not exist yet.
- The **makedirs()** method creates both a directory and all the parents of the directory.

Following example creates "/tmp/user/java/bin" directory:

```

import java.io.File;

public class CreateDir {
    public static void main(String args[]) {
        String dirname = "/tmp/user/java/bin";
        File d = new File(dirname);
        // Create directory now.
        d.mkdirs();
    }
}

```

```
}  
}
```

Compile and execute above code to create "/tmp/user/java/bin".

Note: Java automatically takes care of path separators on UNIX and Windows as per conventions. If you use a forward slash (/) on a Windows version of Java, the path will still resolve correctly.

Listing Directories:

You can use **list()** method provided by **File** object to list down all the files and directories available in a directory as follows:

```
import java.io.File;  
  
public class ReadDir {  
    public static void main(String[] args) {  
  
        File file = null;  
        String[] paths;  
  
        try{  
            // create new file object  
            file = new File("/tmp");  
  
            // array of files and directory  
            paths = file.list();  
  
            // for each name in the path array  
            for(String path:paths)  
            {  
                // prints filename and directory name  
                System.out.println(path);  
            }  
        } catch(Exception e){  
            // if any error occurs  
            e.printStackTrace();  
        }  
    }  
}
```

This would produce following result based on the directories and files available in your **/tmp** directory:

```
test1.txt  
test2.txt  
ReadDir.java  
ReadDir.class
```

Serialization:-

Java provides a mechanism, called object serialization where an object can be represented as a sequence of bytes that includes the object's data as well as information about the object's type and the types of data stored in the object.

After a serialized object has been written into a file, it can be read from the file and deserialized that is, the type information and bytes that represent the object and its data can be used to recreate the object in memory.

Most impressive is that the entire process is JVM independent, meaning an object can be serialized on one platform and deserialized on an entirely different platform.

Classes **ObjectInputStream** and **ObjectOutputStream** are high-level streams that contain the methods for serializing and deserializing an object.

The **ObjectOutputStream** class contains many write methods for writing various data types, but one method in particular stands out:

```
public final void writeObject(Object x) throws IOException
```

The above method serializes an **Object** and sends it to the output stream. Similarly, the **ObjectInputStream** class contains the following method for deserializing an object:

```
public final Object readObject() throws IOException,  
                                ClassNotFoundException
```

This method retrieves the next **Object** out of the stream and deserializes it. The return value is **Object**, so you will need to cast it to its appropriate data type.

To demonstrate how serialization works in Java, I am going to use the **Employee** class that we discussed early on in the book. Suppose that we have the following **Employee** class, which implements the **Serializable** interface:

```
public class Employee implements java.io.Serializable  
{  
    public String name;  
    public String address;  
    public transient int SSN;  
    public int number;  
    public void mailCheck()  
    {  
        System.out.println("Mailing a check to " + name  
                           + " " + address);  
    }  
}
```

Notice that for a class to be serialized successfully, two conditions must be met:

- The class must implement the **java.io.Serializable** interface.
- All of the fields in the class must be serializable. If a field is not serializable, it must be marked **transient**.

If you are curious to know if a Java Standard Class is serializable or not, check the documentation for the class. The test is simple: If the class implements **java.io.Serializable**, then it is serializable; otherwise, it's not.

Serializing an Object:

The **ObjectOutputStream** class is used to serialize an **Object**. The following **SerializeDemo** program instantiates an **Employee** object and serializes it to a file.

When the program is done executing, a file named employee.ser is created. The program does not generate any output, but study the code and try to determine what the program is doing.

Note: When serializing an object to a file, the standard convention in Java is to give the file a **.ser** extension.

```
import java.io.*;

public class SerializeDemo
{
    public static void main(String [] args)
    {
        Employee e = new Employee();
        e.name = "Reyan Ali";
        e.address = "Phokka Kuan, Ambehta Peer";
        e.SSN = 11122333;
        e.number = 101;
        try
        {
            FileOutputStream fileOut =
                new FileOutputStream("/tmp/employee.ser");
            ObjectOutputStream out = new ObjectOutputStream(fileOut);
            out.writeObject(e);
            out.close();
            fileOut.close();
            System.out.printf("Serialized data is saved in /tmp/employee.ser");
        } catch (IOException i)
        {
            i.printStackTrace();
        }
    }
}
```

Deserializing an Object:

The following DeserializeDemo program deserializes the Employee object created in the SerializeDemo program. Study the program and try to determine its output:

```
import java.io.*;

public class DeserializeDemo
{
    public static void main(String [] args)
    {
        Employee e = null;
        try
        {
            FileInputStream fileIn = new FileInputStream("/tmp/employee.ser");
            ObjectInputStream in = new ObjectInputStream(fileIn);
            e = (Employee) in.readObject();
            in.close();
            fileIn.close();
        } catch (IOException i)
        {
            i.printStackTrace();
            return;
        } catch (ClassNotFoundException c)
        {
            System.out.println("Employee class not found");
            return;
        }
        System.out.println("Employee object deserialized successfully");
    }
}
```

```

    {
        System.out.println("Employee class not found");
        c.printStackTrace();
        return;
    }
    System.out.println("Deserialized Employee...");
    System.out.println("Name: " + e.name);
    System.out.println("Address: " + e.address);
    System.out.println("SSN: " + e.SSN);
    System.out.println("Number: " + e.number);
}
}

```

This would produce the following result:

```

Deserialized Employee...
Name: Reyan Ali
Address:Phokka Kuan, Ambehta Peer
SSN: 0
Number:101

```

Here are following important points to be noted:

- The try/catch block tries to catch a `ClassNotFoundException`, which is declared by the `readObject()` method. For a JVM to be able to deserialize an object, it must be able to find the bytecode for the class. If the JVM can't find a class during the deserialization of an object, it throws a `ClassNotFoundException`.
- Notice that the return value of `readObject()` is cast to an `Employee` reference.
- The value of the SSN field was 11122333 when the object was serialized, but because the field is transient, this value was not sent to the output stream. The SSN field of the deserialized `Employee` object is 0.

CHAPTER 17

JDBC

What is JDBC?

JDBC stands for **Java Database Connectivity**, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating SQL or MySQL statements
- Executing that SQL or MySQL queries in the database
- Viewing & Modifying the resulting records

Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database. Java can be used to write different types of executables, such as:

- Java Applications
- Java Applets
- Java Servlets
- Java ServerPages (JSPs)
- Enterprise JavaBeans (EJBs)

All of these different executables are able to use a JDBC driver to access a database and take advantage of the stored data.

JDBC provides the same capabilities as ODBC, allowing Java programs to contain database-independent code.

Pre-Requisite:

Before progressing on this tutorial you need to have good understanding on the following two subjects:

- [Core JAVA Programming](#)
- [SQL or MySQL Database](#)

JDBC Architecture:

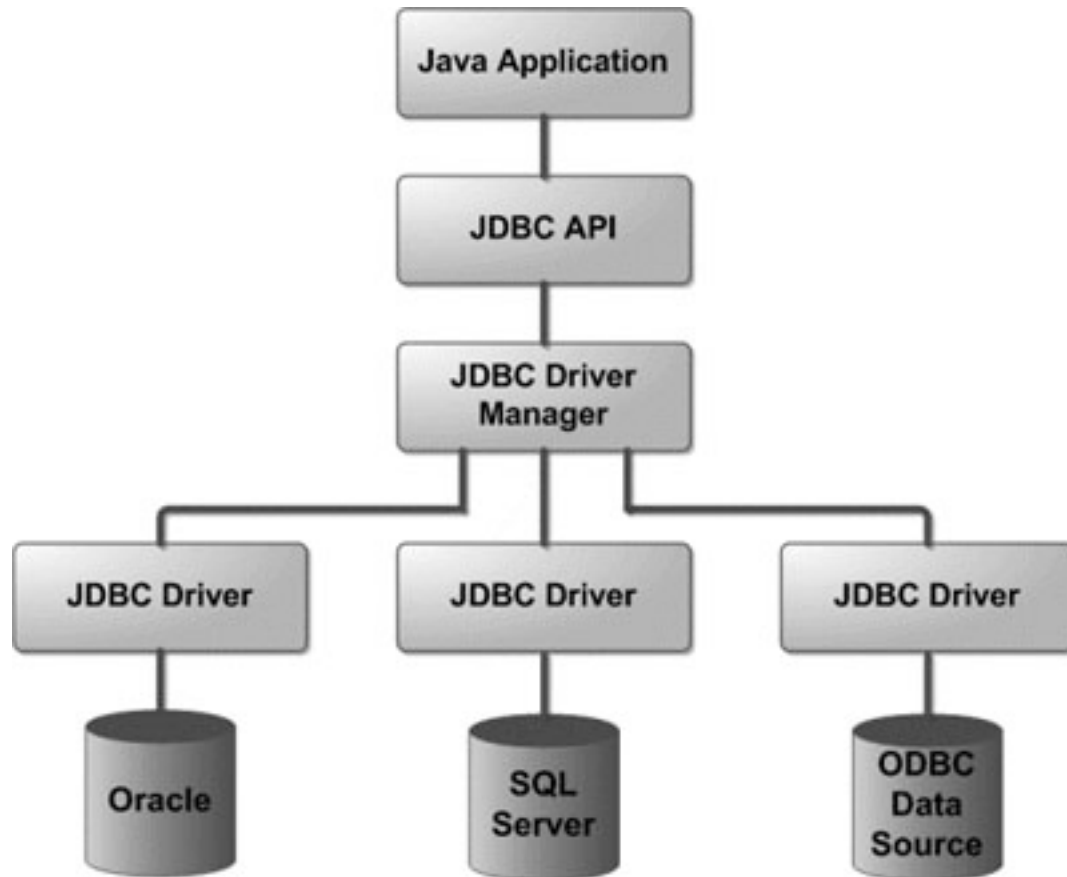
The JDBC API supports both two-tier and three-tier processing models for database access but in general JDBC Architecture consists of two layers:

- **JDBC API:** This provides the application-to-JDBC Manager connection.
- **JDBC Driver API:** This supports the JDBC Manager-to-Driver Connection.

The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.

The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application:



Common JDBC Components:

The JDBC API provides the following interfaces and classes:

- **DriverManager:** This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication subprotocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.
- **Driver:** This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects
- **Connection :** This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement :** You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet:** These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
- **SQLException:** This class handles any errors that occur in a database application.

JDBC Driver :-

JDBC drivers implement the defined interfaces in the JDBC API for interacting with your database server.

For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.

The *Java.sql* package that ships with JDK contains various classes with their behaviours defined and their actual implementations are done in third-party drivers. Third party vendors implements the *java.sql.Driver* interface in their database driver.

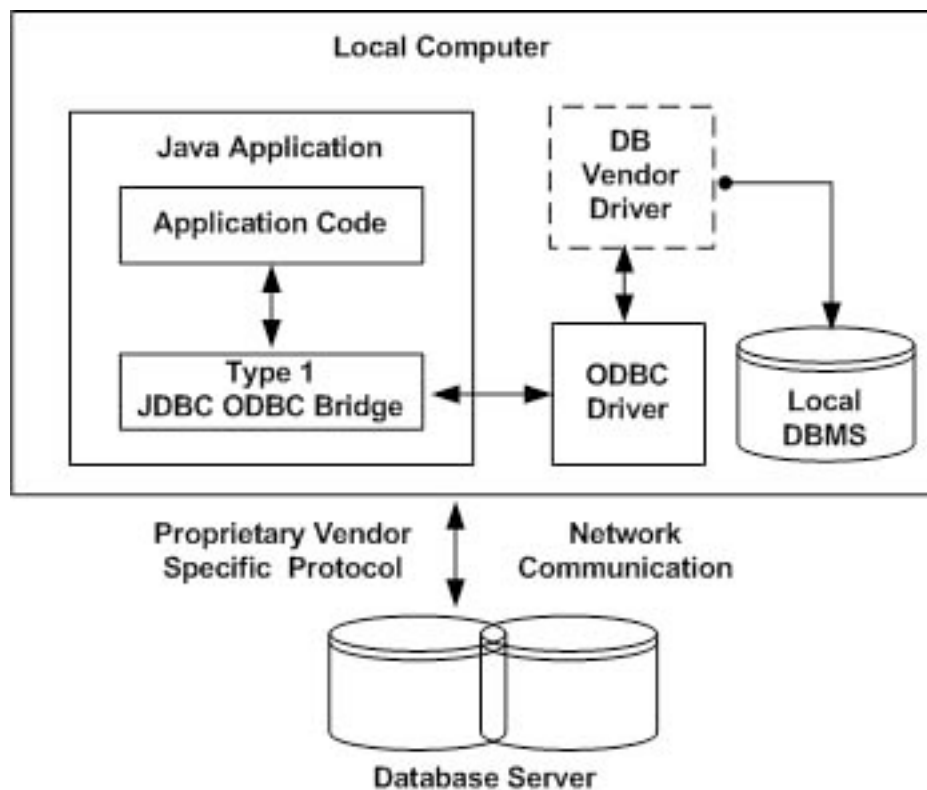
JDBC Drivers Types:

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3, and 4, which is explained below:

Type 1: JDBC-ODBC Bridge Driver:

In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC requires configuring on your system a Data Source Name (DSN) that represents the target database.

When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.

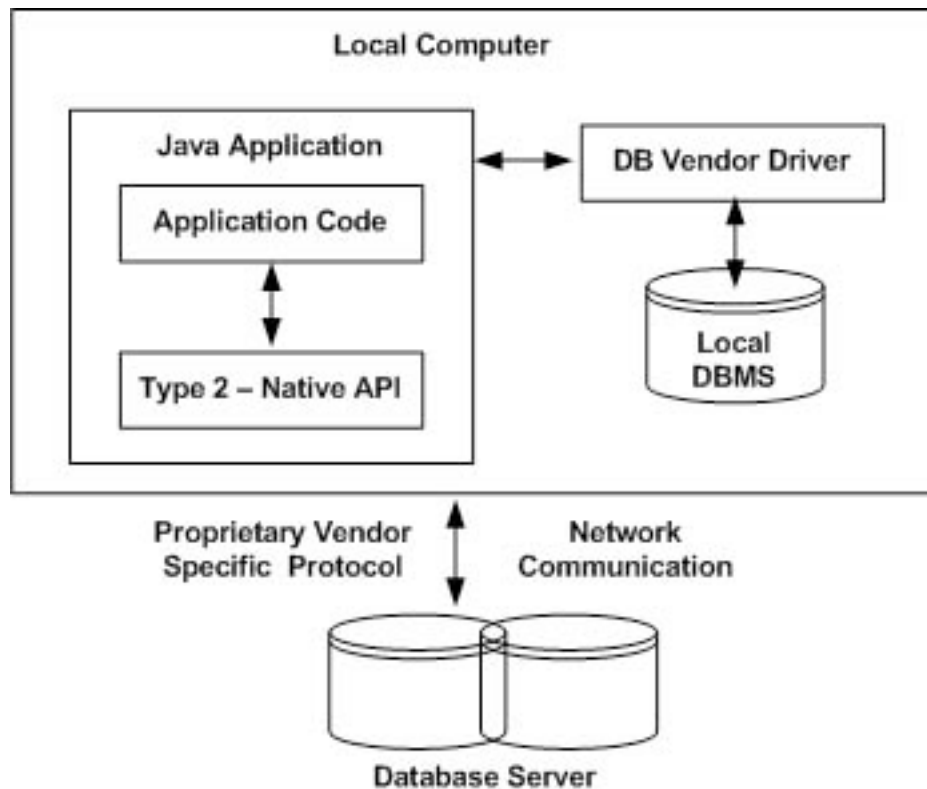


The JDBC-ODBC bridge that comes with JDK 1.2 is a good example of this kind of driver.

Type 2: JDBC-Native API:

In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls which are unique to the database. These drivers typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge, the vendor-specific driver must be installed on each client machine.

If we change the Database we have to change the native API as it is specific to a database and they are mostly obsolete now but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.

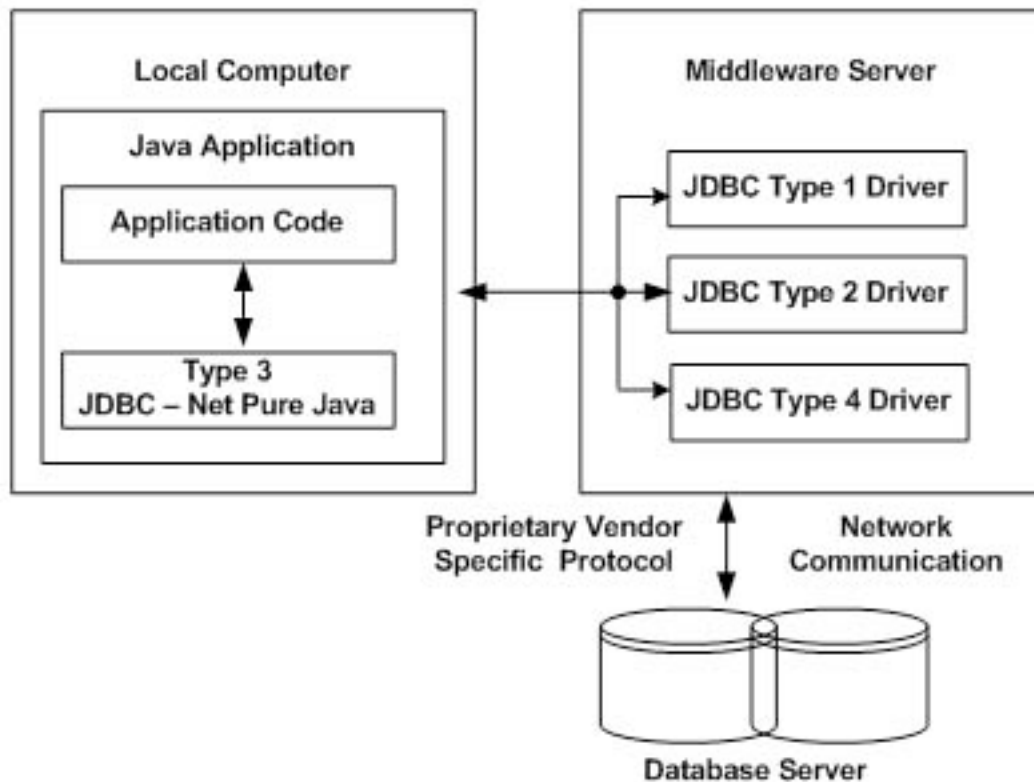


The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

Type 3: JDBC-Net pure Java:

In a Type 3 driver, a three-tier approach is used to accessing databases. The JDBC clients use standard network sockets to communicate with an middleware application server. The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server.

This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



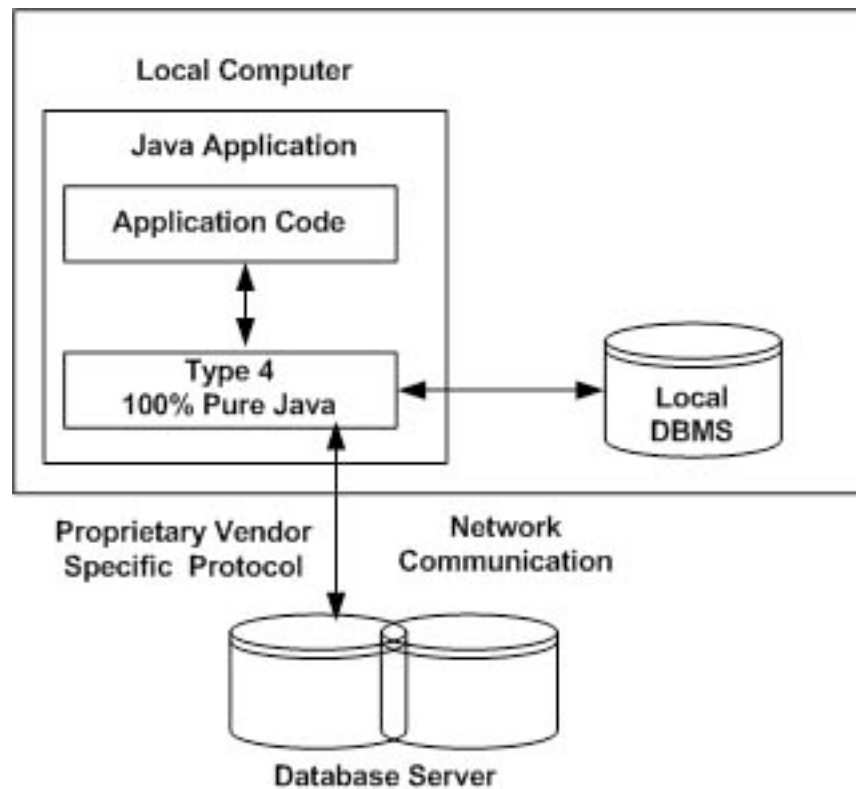
You can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, you need some knowledge of the application server's configuration in order to effectively use this driver type.

Your application server might use a Type 1, 2, or 4 driver to communicate with the database, understanding the nuances will prove helpful.

Type 4: 100% pure Java:

In a Type 4 driver, a pure Java-based driver that communicates directly with vendor's database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself.

This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.



MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

Which Driver should be used?

If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.

If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.

Type 2 drivers are useful in situations where a type 3 or type 4 driver is not available yet for your database.

The type 1 driver is not considered a deployment-level driver and is typically used for development and testing purposes only.

CHAPTER 18

NETWORKING

The term *network programming* refers to writing programs that execute across multiple devices (computers), in which the devices are all connected to each other using a network.

The java.net package of the J2SE APIs contains a collection of classes and interfaces that provide the low-level communication details, allowing you to write programs that focus on solving the problem at hand.

The java.net package provides support for the two common network protocols:

- **TCP:** TCP stands for Transmission Control Protocol, which allows for reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.
- **UDP:** UDP stands for User Datagram Protocol, a connection-less protocol that allows for packets of data to be transmitted between applications.

Socket Programming:

Sockets provide the communication mechanism between two computers using TCP. A client program creates a socket on its end of the communication and attempts to connect that socket to a server.

When the connection is made, the server creates a socket object on its end of the communication. The client and server can now communicate by writing to and reading from the socket.

The java.net.Socket class represents a socket, and the java.net.ServerSocket class provides a mechanism for the server program to listen for clients and establish connections with them.

The following steps occur when establishing a TCP connection between two computers using sockets:

- The server instantiates a ServerSocket object, denoting which port number communication is to occur on.
- The server invokes the accept() method of the ServerSocket class. This method waits until a client connects to the server on the given port.
- After the server is waiting, a client instantiates a Socket object, specifying the server name and port number to connect to.
- The constructor of the Socket class attempts to connect the client to the specified server and port number. If communication is established, the client now has a Socket object capable of communicating with the server.
- On the server side, the accept() method returns a reference to a new socket on the server that is connected to the client's socket.

After the connections are established, communication can occur using I/O streams. Each socket has both an OutputStream and an InputStream. The client's OutputStream is connected to the server's InputStream, and the client's InputStream is connected to the server's OutputStream.

TCP is a twoway communication protocol, so data can be sent across both streams at the same time. There are following usefull classes providing complete set of methods to implement sockets.

ServerSocket Class Methods:

The **java.net.ServerSocket** class is used by server applications to obtain a port and listen for client requests

The ServerSocket class has four constructors:

SN	Methods with Description
1	public ServerSocket(int port) throws IOException Attempts to create a server socket bound to the specified port. An exception occurs if the port is already bound by another application.
2	public ServerSocket(int port, int backlog) throws IOException Similar to the previous constructor, the backlog parameter specifies how many incoming clients to store in a wait queue.
3	public ServerSocket(int port, int backlog, InetAddress address) throws IOException Similar to the previous constructor, the InetAddress parameter specifies the local IP address to bind to. The InetAddress is used for servers that may have multiple IP addresses, allowing the server to specify which of its IP addresses to accept client requests on
4	public ServerSocket() throws IOException Creates an unbound server socket. When using this constructor, use the bind() method when you are ready to bind the server socket

If the ServerSocket constructor does not throw an exception, it means that your application has successfully bound to the specified port and is ready for client requests.

Here are some of the common methods of the ServerSocket class:

SN	Methods with Description
1	public int getLocalPort() Returns the port that the server socket is listening on. This method is useful if you passed in 0 as the port number in a constructor and let the server find a port for you.
2	public Socket accept() throws IOException Waits for an incoming client. This method blocks until either a client connects to the server on the specified port or the socket times out, assuming that the time-out value has been set using the setSoTimeout() method. Otherwise, this method blocks indefinitely
3	public void setSoTimeout(int timeout) Sets the time-out value for how long the server socket waits for a client during the accept().
4	public void bind(SocketAddress host, int backlog) Binds the socket to the specified server and port in the SocketAddress object. Use this method if you instantiated the ServerSocket using the no-argument constructor.

When the ServerSocket invokes accept(), the method does not return until a client connects. After a client does connect, the ServerSocket creates a new Socket on an unspecified port and returns a reference to this new Socket. A TCP connection now exists between the client and server, and communication can begin.

Socket Class Methods:

The **java.net.Socket** class represents the socket that both the client and server use to communicate with each other. The client obtains a Socket object by instantiating one, whereas the server obtains a Socket object from the return value of the accept() method.

The Socket class has five constructors that a client uses to connect to a server:

SN	Methods with Description
1	public Socket(String host, int port) throws UnknownHostException, IOException. This method attempts to connect to the specified server at the specified port. If this constructor does not throw an exception, the connection is successful and the client is connected to the server.
2	public Socket(InetAddress host, int port) throws IOException This method is identical to the previous constructor, except that the host is denoted by an InetAddress object.
3	public Socket(String host, int port, InetAddress localAddress, int localPort) throws IOException. Connects to the specified host and port, creating a socket on the local host at the specified address and port.
4	public Socket(InetAddress host, int port, InetAddress localAddress, int localPort) throws IOException. This method is identical to the previous constructor, except that the host is denoted by an InetAddress object instead of a String
5	public Socket() Creates an unconnected socket. Use the connect() method to connect this socket to a server.

When the Socket constructor returns, it does not simply instantiate a Socket object but it actually attempts to connect to the specified server and port.

Some methods of interest in the Socket class are listed here. Notice that both the client and server have a Socket object, so these methods can be invoked by both the client and server.

SN	Methods with Description
1	public void connect(SocketAddress host, int timeout) throws IOException This method connects the socket to the specified host. This method is needed only when you instantiated the Socket using the no-argument constructor.
2	public InetAddress getInetAddress() This method returns the address of the other computer that this socket is connected to.
3	public int getPort() Returns the port the socket is bound to on the remote machine.
4	public int getLocalPort() Returns the port the socket is bound to on the local machine.
5	public SocketAddress getRemoteSocketAddress() Returns the address of the remote socket.
6	public InputStream getInputStream() throws IOException Returns the input stream of the socket. The input stream is connected to the output stream of the remote socket.
7	public OutputStream getOutputStream() throws IOException Returns the output stream of the socket. The output stream is connected to the input stream of the remote socket
8	public void close() throws IOException Closes the socket, which makes this Socket object no longer capable of connecting again to any server

InetAddress Class Methods:

This class represents an Internet Protocol (IP) address. Here are following usefull methods which you would need while doing socket programming:

SN	Methods with Description
1	static InetAddress getByAddress(byte[] addr) Returns an InetAddress object given the raw IP address .
2	static InetAddress getByAddress(String host, byte[] addr)

	Create an InetAddress based on the provided host name and IP address.
3	static InetAddress getByName(String host) Determines the IP address of a host, given the host's name.
4	String getHostAddress() Returns the IP address string in textual presentation.
5	String getHostName() Gets the host name for this IP address.
6	static InetAddress InetAddress getLocalHost() Returns the local host.
7	String toString() Converts this IP address to a String.

Socket Client Example:

The following GreetingClient is a client program that connects to a server by using a socket and sends a greeting, and then waits for a response.

// File Name GreetingClient.java

```
import java.net.*;
import java.io.*;

public class GreetingClient
{
    public static void main(String [] args)
    {
        String serverName = args[0];
        int port = Integer.parseInt(args[1]);
        try
        {
            System.out.println("Connecting to " + serverName
                               + " on port " + port);
            Socket client = new Socket(serverName, port);
            System.out.println("Just connected to "
                               + client.getRemoteSocketAddress());
            OutputStream outToServer = client.getOutputStream();
            DataOutputStream out =
                new DataOutputStream(outToServer);

            out.writeUTF("Hello from "
                        + client.getLocalSocketAddress());
            InputStream inFromServer = client.getInputStream();
            DataInputStream in =
                new DataInputStream(inFromServer);
            System.out.println("Server says " + in.readUTF());
            client.close();
        } catch (IOException e)
        {
            e.printStackTrace();
        }
    }
}
```

Socket Server Example:

The following GreetingServer program is an example of a server application that uses the Socket class to listen for clients on a port number specified by a command-line argument:

```
// File Name GreetingServer.java

import java.net.*;
import java.io.*;

public class GreetingServer extends Thread
{
    private ServerSocket serverSocket;

    public GreetingServer(int port) throws IOException
    {
        serverSocket = new ServerSocket(port);
        serverSocket.setSoTimeout(10000);
    }

    public void run()
    {
        while(true)
        {
            try
            {
                System.out.println("Waiting for client on port " +
                    serverSocket.getLocalPort() + "...");
                Socket server = serverSocket.accept();
                System.out.println("Just connected to "
                    + server.getRemoteSocketAddress());
                DataInputStream in =
                    new DataInputStream(server.getInputStream());
                System.out.println(in.readUTF());
                DataOutputStream out =
                    new DataOutputStream(server.getOutputStream());
                out.writeUTF("Thank you for connecting to "
                    + server.getLocalSocketAddress() + "\nGoodbye!");
                server.close();
            } catch(SocketTimeoutException s)
            {
                System.out.println("Socket timed out!");
                break;
            } catch(IOException e)
            {
                e.printStackTrace();
                break;
            }
        }
    }

    public static void main(String [] args)
    {
        int port = Integer.parseInt(args[0]);
        try
        {

```

```

        Thread t = new GreetingServer(port);
        t.start();
    } catch (IOException e)
    {
        e.printStackTrace();
    }
}
}

```

Compile client and server and then start server as follows:

```

$ java GreetingServer 6066
Waiting for client on port 6066...

```

Check client program as follows:

```

$ java GreetingClient localhost 6066
Connecting to localhost on port 6066
Just connected to localhost/127.0.0.1:6066
Server says Thank you for connecting to /127.0.0.1:6066
Goodbye!

```

Java URL:-

The **Java URL** class represents an URL. URL is an acronym for Uniform Resource Locator. It points to a resource on the World Wide Web. For example:

1. <http://www.javatpoint.com/java-tutorial>

A URL contains many information:

1. **Protocol:** In this case, http is the protocol.
2. **Server name or IP Address:** In this case, www.javatpoint.com is the server name.
3. **Port Number:** It is an optional attribute. If we write <http://www.javatpoint.com:80/sonoojaiswal/>, 80 is the port number. If port number is not mentioned in the URL, it returns -1.
4. **File Name or directory name:** In this case, index.jsp is the file name.

Commonly used methods of Java URL class:-

The java.net.URL class provides many methods. The important methods of URL class are given below.

Method	Description
public String getProtocol()	it returns the protocol of the URL.
public String getHost()	it returns the host name of the URL.
public String getPort()	it returns the Port Number of the URL.

public String getFile()	it returns the file name of the URL.
public URLConnection.openConnection()	it returns the instance of URLConnection i.e. associated with this URL.

Example of Java URL class

```

1. //URLDemo.java
2. import java.io.*;
3. import java.net.*;
4. public class URLDemo{
5.     public static void main(String[] args){
6.         try{
7.             URL url=new URL("http://www.javatpoint.com/java-tutorial");
8.
9.             System.out.println("Protocol: "+url.getProtocol());
10.            System.out.println("Host Name: "+url.getHost());
11.            System.out.println("Port Number: "+url.getPort());
12.            System.out.println("File Name: "+url.getFile());
13.
14.        }catch(Exception e){System.out.println(e);}
15.    }
16. }
```

Output:

```

Protocol: http
Host Name: www.javatpoint.com
Port Number: -1
File Name: /java-tutorial
```

Java URLConnection class:-

The **Java URLConnection** class represents a communication link between the URL and the application. This class can be used to read and write data to the specified resource referred by the URL.

How to get the object of URLConnection class

The `openConnection()` method of `URL` class returns the object of `URLConnection` class. Syntax:

```
1. public URLConnection openConnection()throws IOException{}
```

Displaying source code of a webpage by URLConnecton class:-

The `URLConnection` class provides many methods, we can display all the data of a webpage by using the `getInputStream()` method. The `getInputStream()` method returns all the data of the specified URL in the stream that can be read and displayed.

Example of Java URLConnecton class

```
1. import java.io.*;
2. import java.net.*;
3. public class URLConnectionExample {
4.     public static void main(String[] args){
5.         try{
6.             URL url=new URL("http://www.javatpoint.com/java-tutorial");
7.             URLConnection urlcon=url.openConnection();
8.             InputStream stream=urlcon.getInputStream();
9.             int i;
10.            while((i=stream.read())!=-1){
11.                System.out.print((char)i);
12.            }
13.        }catch(Exception e){System.out.println(e);}
14.    }
15. }
```